A PBL-III (AIM3170) Synopsis on

PACE

Project Allocation, Coordination and Evaluation Portal

Submitted to Manipal University Jaipur
Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

In Computer Science and Engineering (AIML)

2026-2027

By

Mihir Sanghvi 2427010544
Krishna Agarwal 2427010030
Mahika Tuteja 2427010439



MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

Dr. Preeti Narooka
Team Supervisor

Department of Artificial Intelligence and Machine Learning
Manipal University Jaipur
Jaipur, Rajasthan

I. INTRODUCTION TO PROBLEM

Type of project: Application based (a web application for students, team supervisors and the PBL office).

What is the problem?

Project-Based Learning (PBL) asks every student team to form a team, find a faculty supervisor, register its project with a signed PBL form, report its progress every week and be graded on it. In our department this cycle is still coordinated through paper forms, email threads, WhatsApp messages and shared spreadsheets. A team lead walks a signed form from one office to another; a supervisor receives weekly reports as attachments scattered across an inbox; the PBL office collects lists of teams by hand to find out which students still have no team and which teams still have no supervisor.

Why does it need to be solved?

Each of these steps is small, but together they cost time and cause real problems. Signed forms go missing, students do not know whether their supervisor request has been seen, a supervisor can end up with more groups than allowed, a student can appear in two teams, and ungraded weekly reports are easy to forget. Research on PBL reports better engagement and deeper learning when students own a real project [1], [2], but also notes that PBL demands much more coordination from teachers than lecture-based teaching [3]. General learning management systems such as Google Classroom or Moodle handle assignments and grades well, but none of them model the PBL-specific steps: team requests, choosing a supervisor with a signed form, supervisor capacity, and an office-wide view of unallocated students.

How do we propose to solve it?

We propose PACE (Project Allocation, Coordination and Evaluation), a web portal that carries the whole PBL cycle in one place for three roles:

- **Student:** sign up with a verified MUJ email, form a team by sending team requests, send the signed PBL form to a chosen supervisor, submit weekly reports, see grades and feedback, raise queries and file achievements.
- **Team Supervisor:** approve, send back or reject mentor requests, read and grade weekly reports, grade each student with improvement notes, answer queries and post announcements.
- **Administrator (PBL office):** see every group and every student without a group, allot or move supervisors, and get a shortlist for the Student Excellence Award.

PACE is built with Next.js and React on the front end and Supabase (PostgreSQL, Auth and Storage) on the back end, and is hosted on Vercel. The key design idea is to place every PBL rule (team size, one group per student, supervisor limit, who may grade) inside the database as rule-checking functions protected by Row Level Security, so that no mistake in the user interface can break a rule. Every request, decision, report and grade automatically notifies the people involved, so each user always sees one clear next step.

II. LITERATURE SURVEY

We studied seven existing software tools and methods that institutions and student teams use today to run project work. For each we note what it is, along with its pros and cons when used for PBL.

1. Google Classroom [4]

A free class management tool from Google Workspace for Education used for posting assignments, collecting work and returning grades.

Pros:

- Free for institutions and very easy to use.
- Good assignment, rubric and grading support, with Google Drive integration.

Cons:

- No concept of teams choosing a supervisor or signed registration forms.
- Groups must be managed by hand; no limit on how many teams a teacher takes.
- No office-wide view of students without a team.

2. Moodle LMS [5]

An open-source learning management system widely used by universities, with assignment, group and workshop (peer review) activities.

Pros:

- Open source and highly configurable through plugins.
- Group submissions, grading and detailed reports.

Cons:

- Self-hosting and plugin setup need dedicated staff.
- Interface is heavy for a simple PBL flow.
- Supervisor selection and approval would need custom development.

3. Microsoft Teams for Education [6]

A collaboration platform with chat, meetings, files and an Assignments app, often used for class communication.

Pros:

- Strong chat, meetings and file sharing in one place.
- Assignments with rubrics and grading.

Cons:

- Communication-centric; PBL steps get lost in chat threads.
- No team formation rules, supervisor capacity or registration workflow.

- Depends on institutional Microsoft 365 licences.

4. Canvas LMS [7]

A learning management system that supports student self sign-up groups with size limits, group assignments and SpeedGrader.

Pros:

- Self sign-up groups with a maximum size and a list of unassigned students.
- Mature grading tools and a clean interface.

Cons:

- Hosted version is paid; self-hosting is complex.
- Groups are per course, not tied to a supervisor chosen by the team.
- No signed-form approval or supervisor limit.

5. GitHub Classroom [8]

A free tool that creates repositories for individual and group assignments and can run automatic tests on code.

Pros:

- Team assignments with a maximum team size.
- Version control and autograding for code projects.

Cons:

- Suited only to software projects and to students who know Git.
- No supervisor selection, weekly reporting or feedback workflow.
- Teachers manage each classroom separately.

6. Jira and Trello [9]

Project management tools for planning tasks on boards and tracking their progress, used by many student teams informally.

Pros:

- Excellent task tracking, boards and progress views.
- Free plans for small teams.

Cons:

- Built for industry teams, not for academic evaluation.
- No grading, supervisor registration or institution-wide view.
- Each team sets up its own board, so the PBL office sees nothing.

7. Student-Project Allocation algorithms [10]

Research methods that match students to projects and supervisors using preference lists while respecting capacity limits (the SPA problem).

Pros:

- Provably stable matchings that respect supervisor capacity.
- Fair when every student and supervisor submits preferences.

Cons:

- Allocation only; they do not cover reporting, grading or communication.
- Need complete preference lists up front, which does not fit teams that talk to supervisors in person first.

Summary of the survey

General learning management systems (Google Classroom, Moodle, Microsoft Teams and Canvas) are strong at assignments and grading but do not model how a PBL team forms and registers with a supervisor. Project tools (Jira, Trello, GitHub Classroom) help a team work but give the institution no view or control. Allocation algorithms solve matching but nothing after it. This gap, a single portal that enforces the PBL rules from team formation to final grade, is what PACE addresses.

III. COMPARATIVE STUDY

Table 1 compares the surveyed tools with PACE on the features a PBL cycle needs. "Partly" means the feature exists but needs manual work, plugins or a paid plan to fit PBL.

Table 1: Comparison of existing tools with PACE

Feature	Google Classroom	Moodle	MS Teams	Canvas	GitHub Classroom	Jira / Trello	PACE
Team formation by requests with size limit	No	Partly	No	Yes	Yes	No	Yes
Supervisor chosen by team with signed form	No	No	No	No	No	No	Yes
Supervisor approve / ask for changes / reject	No	No	No	No	No	No	Yes
Limit on groups per supervisor	No	No	No	No	No	No	Yes
Weekly report submission and grading	Yes	Yes	Yes	Yes	Partly	No	Yes
Individual grades with improvement notes	Yes	Yes	Yes	Yes	Partly	No	Yes
Office view of students without a team	No	Partly	No	Partly	No	No	Yes
Automatic notifications for every step	Yes	Yes	Yes	Yes	Partly	Yes	Yes
Achievements with proof and LinkedIn posting	No	No	No	No	No	No	Yes
Task tracking inside the project	No	Partly	Partly	No	Partly	Yes	Partly
Rules enforced by the database	No	No	No	No	No	No	Yes
Free to run for a department	Yes	Yes	Partly	Partly	Yes	Partly	Yes

The comparison shows that no existing tool covers the PBL-specific steps (supervisor registration, supervisor limits and an office-wide view), while PACE covers them alongside the reporting and grading features that learning management systems already offer. PACE does not aim to replace a full task tracker; teams can keep using one for day-to-day work.

IV. OBJECTIVES

1. To design and develop a web portal that carries the complete PBL cycle (team formation, supervisor registration, weekly reporting, grading, queries and announcements) for students, team supervisors and the administrator in one place.
2. To enforce the PBL rules (at most three members per team, one group per student, at most seven groups per supervisor, and grading only by the group's own supervisor) inside the database using rule-checking functions and Row Level Security, so that no client can bypass them.
3. To give every role a dashboard with one clear next step and automatic notifications, so that no request, report or grade is forgotten.
4. To give the PBL office a live view of every group and every student without a group, with tools to allot or move supervisors and to shortlist students for the Student Excellence Award.
5. To keep the system secure and free to run for a department, using university email sign-in, private file storage with expiring links and managed hosting.

V. PLANNING OF WORK

Methodology

The project follows an iterative and incremental methodology. The steps below are followed to achieve the objectives:

Step 1. **Requirement study.** Study the department's PBL guidelines and the paper forms used in earlier semesters, discuss the process with our supervisor, and write functional and non-functional requirements as short, testable statements [11].

Step 2. **System design.** Design the database schema (profiles, students, faculty, groups, members, team requests, mentor requests, weekly reports, grades, achievements, notifications), the Row Level Security policies, the workflows for team formation and mentor registration, and sketches of each screen.

Step 3. **Technology selection.** Choose a stack that a small team can build and run for free: Next.js and React with TypeScript for the front end [12], [13], Supabase for database, authentication and storage [14], and Vercel for hosting.

Step 4. **Iterative development.** Build the portal in one-week increments following an agile approach [15]. Each week starts with a short plan agreed with the supervisor and ends with a working, deployed version and a weekly progress report.

Step 5. **Data preparation.** Scrape the MUJ faculty directory into the database so that supervisors can be searched by name, department and expertise, and prepare test accounts for every role.

Step 6. **Testing.** Test every feature against written test cases with student, supervisor and administrator accounts, and try to break each rule directly (for example, adding a fourth member to a team) to check that the database refuses it.

Step 7. **Deployment and review.** Deploy every increment to Vercel from the main branch, demonstrate it to the supervisor, and fold the feedback into the next week's plan.

Step 8. **Documentation and pilot.** Write the project report and user notes, then plan a pilot of the portal with one section of students.

Tools and technologies

Table 2: Technology stack

Layer	Technology	Purpose
Front end	Next.js 16, React 19, TypeScript	Pages for the three portals and routing
Styling	Tailwind CSS v4	Responsive layout and design tokens

Layer	Technology	Purpose
Authentication	Supabase Auth	University email sign-up and verification
Database	PostgreSQL (Supabase)	Data, Row Level Security, rule-checking functions
File storage	Supabase Storage	Signed forms, reports and proof files
Hosting	Vercel	Automatic build and deployment
Data import	Python	Scraping the MUJ faculty directory

Proposed system architecture

PACE is a client-server web application (Figure 1). The browser runs the Next.js application for the three portals; every data call goes through a single repository layer to Supabase, where Row Level Security [16] and rule-checking functions decide what each user can read and change. A small set of server routes on Vercel handles LinkedIn posting, because they hold a secret key.

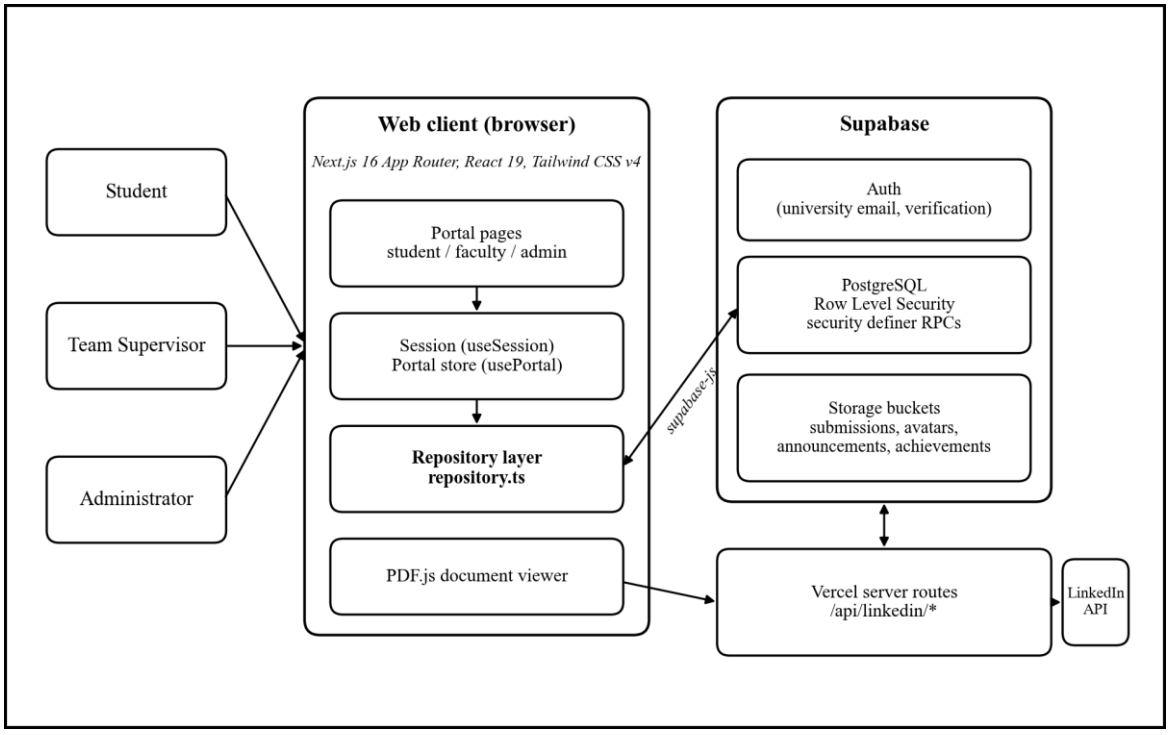


Figure 1: Proposed system architecture of PACE

Work plan

Table 3 shows the week-by-week plan. Weeks I to VII form Phase 1; Figure 2 shows the share of work completed at the end of each week. The remaining items form Phase 2.

Table 3: Weekly work plan

Week	Planned work	Outcome
I	Problem study, requirements, choice of technology stack, interface sketches	Requirements and sketches
II	Database design, landing and sign-in screens, portal layout	Schema and screen layout
III	Next.js project and Vercel deployment, sign-in, student dashboard, project and profile pages	First deployed version
IV	Supabase back end; groups, weekly reports, grading, queries; faculty directory; email-verified sign-up	Core PBL data live
V	Mentor registration with signed form; change of supervisor; database notifications	Supervisor workflow
VI	Achievements; LinkedIn posting from a central account; drag-and-drop uploads	Achievements module
VII	Team requests; supervisor-only grading; formatted announcements; test accounts; report	Phase 1 complete
VIII onwards	University single sign-on, password recovery, evaluation rubrics, at-risk team analytics, pilot	Phase 2

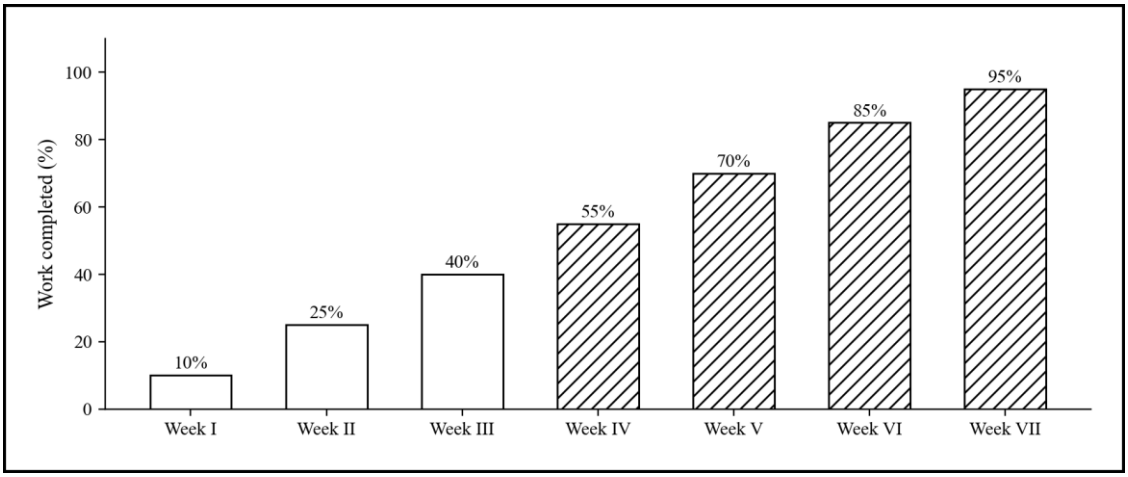


Figure 2: Work completed at the end of each week of Phase 1

Expected outcome

A deployed, secure portal in which students, team supervisors and the PBL office complete the whole PBL cycle without paper forms or email threads, with every PBL rule enforced by the database and every role shown its next step.

VI. BIBLIOGRAPHY / REFERENCES

[1] J. W. Thomas, “A review of research on project-based learning,” The Autodesk Foundation, San Rafael, CA, USA, 2000.

[2] D. Kokotsaki, V. Menzies and A. Wiggins, “Project-based learning: A review of the literature,” Improving Schools, vol. 19, no. 3, pp. 267-282, 2016.

[3] J. S. Krajcik and P. C. Blumenfeld, “Project-based learning,” in The Cambridge Handbook of the Learning Sciences, R. K. Sawyer, Ed. Cambridge, U.K.: Cambridge University Press, 2006, pp. 317-334.

[4] Google Classroom Help, Google, support.google.com/edu/classroom, Last accessed 29 October 2026.

[5] Moodle Documentation, Moodle Pty Ltd, docs.moodle.org, Last accessed 29 October 2026.

[6] Microsoft Teams for Education, Microsoft Learn, learn.microsoft.com/microsoftteams, Last accessed 29 October 2026.

[7] Canvas LMS Guides, Instructure, community.canvaslms.com, Last accessed 29 October 2026.

[8] GitHub Classroom Documentation, GitHub, docs.github.com/education, Last accessed 29 October 2026.

[9] Jira and Trello Documentation, Atlassian, support.atlassian.com, Last accessed 29 October 2026.

[10] D. J. Abraham, R. W. Irving and D. F. Manlove, “Two algorithms for the Student-Project Allocation problem,” Journal of Discrete Algorithms, vol. 5, no. 1, pp. 73-90, 2007.

[11] I. Sommerville, “Software Engineering,” Pearson, 10th ed., ISBN 978-0133943030, 2016.

[12] Next.js Documentation, Vercel, nextjs.org/docs, Last accessed 29 October 2026.

[13] React Documentation, Meta Open Source, react.dev, Last accessed 29 October 2026.

[14] Supabase Documentation: Database, Auth and Storage, Supabase, supabase.com/docs, Last accessed 29 October 2026.

[15] K. Beck et al., Manifesto for Agile Software Development, agilemanifesto.org, Last accessed 29 October 2026.

[16] PostgreSQL Documentation: Row Security Policies, The PostgreSQL Global Development Group, postgresql.org/docs, Last accessed 29 October 2026.